

A partir de la información consolidada en los informes, he diseñado un plan de arquitectura técnico, exhaustivo y profundo para tu aplicación de escritorio. Este diseño está estructurado en una **arquitectura de dos capas** 1 centrada en la filosofía de "offline-first" y soberanía de datos 2, 3, utilizando herramientas multiplataforma y de código abierto. A continuación, presento el plan arquitectónico detallado:

1. Visión General y Stack Tecnológico

El sistema no escribirá directamente en Money Manager Ex (MMEX) desde el inicio para evitar la corrupción de datos y la pérdida de trazabilidad 1, 4. Se utilizará un flujo de Extracción, Transformación y Carga (ETL) local coordinado por un planificador de tareas.

Stack Tecnológico Principal:

- **Lenguaje Base:** Python 3.12 (por su compatibilidad con automatización y manipulación de datos) 5, 6.
- **Base de Datos:** SQLite nativo (sqlite3 en Python) 5, 7.
- **Ingesta Web/API:** playwright (en modo *headful* para evadir bloqueos bancarios chilenos), google-api-python-client (Gmail API) 5.
- **Procesamiento de Texto/Email:** beautifulsoup4 (limpieza HTML), mailbox/imaplib, modelos LLM locales usando Ollama (ej. modelo Qwen2.5 7B) 5, 8-11.
- **Procesamiento de Archivos (CSV, PDF, OFX):** pandas (CSV), ofxtools (OFX/QIF), pdfplumber o PyMuPDF (PDF nativo), y pytesseract / OCRmyPDF (escaneos/imágenes) 5, 12-15.
- **Normalización y Deduplicación:** RapidFuzz para coincidencia difusa de textos comerciales 4, 16.
- **Sincronización y Seguridad:** Syncthing (sincronización P2P), restic (backups cifrados), y keyring (integración con el Credential Manager de Windows) 3, 5, 17.

2. Capa 1: Ingesta de Datos (Data Ingestion)

El sistema debe estar preparado para recolectar información desde múltiples fuentes de forma modular 18.

- **Vía 1: Telemetría de Correo (Primaria y Rápida):**
- Se utilizará la API de Gmail localmente (autenticación OAuth vía client_secret.json) o un cliente local como Thunderbird (leyendo archivos mbox offline) para extraer notificaciones de compra 8, 19, 20.
- La extracción del texto se hará limpiando el HTML con BeautifulSoup 21 y extrayendo los datos clave (fecha, monto, comercio) utilizando expresiones regulares (Regex) estructuradas 21, 22 o, para mayor resiliencia ante cambios de formato, mediante un modelo LLM local con prompt de salida JSON estricto 8, 23.
- **Vía 2: Scraping Semi-Automatizado (Para bancos complejos):**
- Para entidades como BancoEstado o Falabella, que bloquean la navegación invisible o exigen Clave Dinámica, se utilizará Playwright en modo *headful* (navegador visible) con persistencia de sesión (storage_state) 23-25. El usuario autoriza el ingreso manualmente una vez por sesión, y el script extrae las tablas HTML 23, 26.
- **Vía 3: Estados de Cuenta Estructurados y Contingencia:**

- El script monitoreará una carpeta local (ej. inbox/bank_exports) y procesará formatos OFX con ofxtools, formatos MT940, o CSVs bancarios utilizando pandas 14, 27-29.
- Si los bancos proveen PDFs, se extraerá el texto directamente con pdfplumber/PyMuPDF si es nativo, o mediante OCR con Tesseract si es escaneado 15, 30.

3. Capa 2: Motor ETL y Staging Database

Todos los datos recolectados aterrizarán en una base de datos SQLite intermedia (staging.db) 1, 4.

- **Estructura del Esquema Canónico (canonical_tx):** Esta tabla conservará información más rica que la de MMEX: tx_uid (ID único), owner (yo/esposa/conjunto), source_type (email, csv, pdf, etc.), amount, currency, merchant_raw, raw_text (texto original del correo), y la fecha de la transacción 31, 32.
- **Normalización y Heurística Cognitiva:** Las fechas se estandarizarán obligatoriamente a formato ISO 8601 4, 33. Mediante RapidFuzz y un diccionario local, el sistema traducirá descripciones sucias (ej. "TDC 4522 RECARGA") a Beneficiarios limpios enlazados automáticamente a Categorías y Subcategorías predefinidas 34, 35.
- **Deduplicación Robusta (El Hash FITID):** Para evitar ingresos duplicados provenientes de un mismo evento capturado por correo y luego por CSV, el script generará un Hash SHA-256 (identificador determinístico) concatenando: **Fecha + Monto Absoluto + Identificador de Tarjeta + Comercio Limpio** 4, 36. Las colisiones se fusionarán dando prioridad a la información del exporte oficial del banco sobre la del correo 37.

4. Capa 3: Integración con Money Manager Ex (.mmb)

La inserción de los datos limpios hacia el archivo contable de la familia se ejecutará en dos fases progresivas según las mejores prácticas arquitectónicas 38, 39.

Fase 3.1: Minimum Viable Product (MVP) - Inserción Vía Archivo

- El motor ETL exportará los registros desde la staging.db hacia un archivo CSV estructurado o un archivo OFX validado 5, 38-40.
- El usuario importará este archivo usando el "Universal CSV Importer" de MMEX. Esto aprovecha la lógica de validación nativa de la app, importando transacciones con el estado "Followup" (Para seguimiento), permitiendo una revisión humana y mitigando errores destructivos 38, 39, 41.

Fase 3.2: Orquestación Avanzada - Escritura Directa en SQLite (ACID)

- Una vez estabilizado el flujo, un script en Python (conector sqlite3) interactuará directamente con el motor relacional de MMEX 7, 42.
- **Validación Estricta y Transacciones ACID:** El script bloqueará la base de datos temporalmente con el comando BEGIN IMMEDIATE 43. Interactuará secuencialmente insertando nuevos comercios en PAYEE_V1, asociando categorías en CATEGORY_V1, y finalmente guardando el movimiento en el núcleo CHECKINGACCOUNT_V1 44, 45. Si hay un solo error en el proceso, se ejecutará un ROLLBACK total para evitar la corrupción del archivo 42, 46.

- **Uso de Campos Personalizados:** El identificador anti-duplicados (Hash FITID) o el texto original del correo se almacenarán en la tabla de extensión CUSTOMFIELDDATA_V1 de MMEX, evitando ensuciar el campo principal de notas y logrando una base inyectable silenciosa 47, 48.

5. Multiusuario, Privacidad y Colaboración Familiar

- **Ecosistema Único:** En lugar de crear múltiples archivos de base de datos, se utilizará una única base central (finanza.mmb) 3, 49.
- **Modelado de Cuentas:** Se crearán entidades separadas (ACCOUNTID) para "Tarjeta Ricardo" y "Tarjeta Laura" 3. Para manejar gastos conjuntos en el hogar, no se crearán categorías duplicadas, sino que se usará el sistema nativo de **Etiquetas (Tags)** (ej. Tag "Gasto Conjunto"), facilitando así reportes por persona 3, 49, 50. Las transferencias entre esposos usarán el esquema interno TOACCOUNTID de MMEX para neutralizar el impacto en el flujo de caja global 45, 51, 52.
- **Sincronización Segura:** Para mantener la independencia de la nube pública, los equipos de los cónyuges mantendrán sincronizada la carpeta local del proyecto utilizando herramientas peer-to-peer como **Syncthing** 3, 8.

6. Ejecución de Tareas, Automatización y Backups

- **Orquestación en Windows:** El sistema se automatizará usando el Programador de Tareas de Windows (*Task Scheduler*) para que los scripts se ejecuten periódicamente en segundo plano (ej. cada 1-6 horas) 23, 53. Se usarán rutas absolutas y comandos wrapper en .bat asegurando un entorno UTF-8 (chcp 65001) 8, 54.
- **Manejo de Credenciales:** Todas las claves, incluyendo la de los bancos para el *scraping* semi-asistido, se gestionarán encriptadas en el entorno local a través del Windows Credential Manager invocado mediante la librería keyring de Python. Nunca deben guardarse contraseñas en texto plano 49, 55.
- **Backups y Contenedores Inmutables:** Previo a cada escritura programática en el archivo principal .mmb, se configurará la creación de una copia .bak inmutable 56. Periódicamente, los archivos deben respaldarse fuera del entorno de operación (cold storage) utilizando herramientas como **restic** (que provee cifrado nativo AES-256 y control de versiones) o guardando todo el directorio de trabajo dentro de un volumen cifrado de **VeraCrypt** 17, 57.